Workshop 1: Image Classification with CIFAR-10

Aims of the Workshop

In this workshop, you will learn how to build, train, and evaluate models for image classification using the CIFAR-10 dataset. We will start with a traditional machine learning method called the Random Forest classifier. (Don't worry, we haven't covered this in the lecture, but I'll provide a simple explanation here. The goal is to use Random Forest as a classical machine learning baseline for comparison with neural networks.) We'll then move on to building a simple Artificial Neural Network (ANN) using Keras. By the end of the session, you will understand how to:

- Load and explore the CIFAR-10 dataset (images and labels).
- Preprocess image data (reshaping and normalization) for different models.
- Train and evaluate a non-deep learning model (Random Forest) on image data.
- Build, compile, and train a neural network (ANN) for the same task using Keras.
- Evaluate model performance using accuracy, confusion matrices, and simple visualizations.
- (Optional) Experiment with hyperparameter tuning or deeper networks to improve performance.

This is a hands-on session, you will be writing and running code in a Jupyter Notebook for each step. Follow the instructions and code snippets, and feel free to try the suggested optional experiments if time permits.

Prerequisites: Basic Python and NumPy, understanding of simple machine deep learning models (from Lecture 1), and familiarity with Jupyter notebooks. We will be using libraries such as numpy, matplotlib, sklearn (scikit-learn), and keras (TensorFlow). Make sure to import the necessary packages as you go along.

Exercise 1: CIFAR-10 Classification with a Non-Deep Learning Model

In the first part of the workshop, we will tackle the image classification task using a traditional machine learning approach, a Random Forest classifier. This will give us a baseline performance to compare against the neural network later.

About the CIFAR-10 dataset: CIFAR-10 is a collection of 60,000 color images (32x32 pixels) in 10 different classes (objects like animals, vehicles, etc.), with 50,000 images for training and 10,000 for testing. Each image is labeled with one of 10 categories. The classes in CIFAR-10 are:

Class	Name
0	airplane
1	automobile
2	bird

Class	Name
3	cat
4	deer
5	dog
6	frog
7	horse
8	ship
9	truck

We will use these class names later for interpretation of results.



Exercise 1a: Loading and Exploring the CIFAR-10 Dataset python

First, we need to load the CIFAR-10 data and understand its structure. Keras provides a convenient way to load this dataset. Run the code below to load the training and test sets and to examine their shapes and a few example images.

```
# Import libraries
  import numpy as np
  from tensorflow.keras.datasets import cifar10

  # Load CIFAR-10 dataset
  (X_train, y_train), (X_test, y_test) = cifar10.load_data()

  print("Training data shape:", X_train.shape, "Labels shape:",
y_train.shape)
  print("Test data shape:", X_test.shape, "Labels shape:", y_test.shape)

  # Print the first 5 training labels
  print("First 5 training labels:", y_train[:5].ravel())
```

When you run this, you should see that `X_train` is of shape (50000, 32, 32, 3) and `y_train` is (50000, 1) (similarly, 10,000 samples for test). This means we have 50,000 training images, each 32 × 32 pixels with 3 color channels (RGB), and 50,000 corresponding labels. The labels are currently given as integers 0 through 9 (each label is in its own array, hence the extra dimension).
Now, let's do a quick exploration:

- Dimensions: Confirm the above shapes and what they mean.

- Value range: Pixel values are typically 0 to 255. We will need to normalize these later.
- Visual check: It's always useful to see some images to understand the task. Let's display a few sample images with their labels.

```
import matplotlib.pyplot as plt

  # Define the class names for reference
  class_names = ["airplane", "automobile", "bird", "cat", "deer",
                 "dog", "frog", "horse", "ship", "truck"]

  # Plot a grid of 5 sample images from the training set
  plt.figure(figsize=(6,2))
  for i in range(5):
      plt.subplot(1, 5, i+1)
      plt.imshow(X_train[i])
      plt.title(class_names[int(y_train[i])])
      plt.axis('off')  # Hide axes ticks
  plt.show()
```

Hint: If you only see a tiny image, remember each image is just 32 × 32 pixels! The labels above each image should give you an idea of what object it is. Feel free to adjust the index or plot more images to explore the dataset.

Exercise 1b: Preprocessing-Reshaping and Normalization

Before feeding the data to our models, we need to preprocess it appropriately:

- Reshape/Flatten: Traditional machine learning models like Random Forests or logistic regression expect input as a flat feature vector rather than a 2D image grid. We will flatten each image (32 × 32 × 3) into a one-dimensional array of length 3072 (since 32323 = 3072 features per image).
- Normalization: Our neural network will train much better if input pixels are normalized to a 0-1 range. We will scale the pixel values (which range from 0 to 255) down to 0.0-1.0 by dividing by 255.

Let's apply these preprocessing steps:

```
# Convert images from [0,255] to [0,1] for numerical stability
    X_train = X_train.astype('float32') / 255.0
    X_test  = X_test.astype('float32') / 255.0

    # Flatten the images into 1D vectors
    X_train_flat = X_train.reshape(X_train.shape[0], -1)  # shape becomes
(50000, 3072)
    X_test_flat  = X_test.reshape(X_test.shape[0], -1)    # shape becomes
(10000, 3072)

    # Also flatten the label arrays from shape (N,1) to (N,) for use in
scikit-learn
    y_train_flat = y_train.ravel()
```

```
y_test_flat = y_test.ravel()

print("After flattening: ", X_train_flat.shape, y_train_flat.shape)
```

After this step, `X_train_flat` should be of shape (50000, 3072) and `y_train_flat` of shape (50000,). Each row of `X_train_flat` is an image unrolled into 3072 features, and `y_train_flat` is the corresponding class label (0-9). We will use these for the Random Forest.

Hint: If memory or speed becomes an issue (training a Random Forest on 50k images can be slow), you can consider using a subset of the training data (for example, the first 10,000 images) to train the traditional model as a quick experiment. This will reduce accuracy but can speed up the demonstration. However, if time allows, using the full training set will give better results.

Exercise 1c: Training and Evaluating a Random Forest Classifier

Now that the data is prepared, let's train a Random Forest classifier as our non-deep learning approach.

Background: What is a "Random Forest"? (Simple explanation)

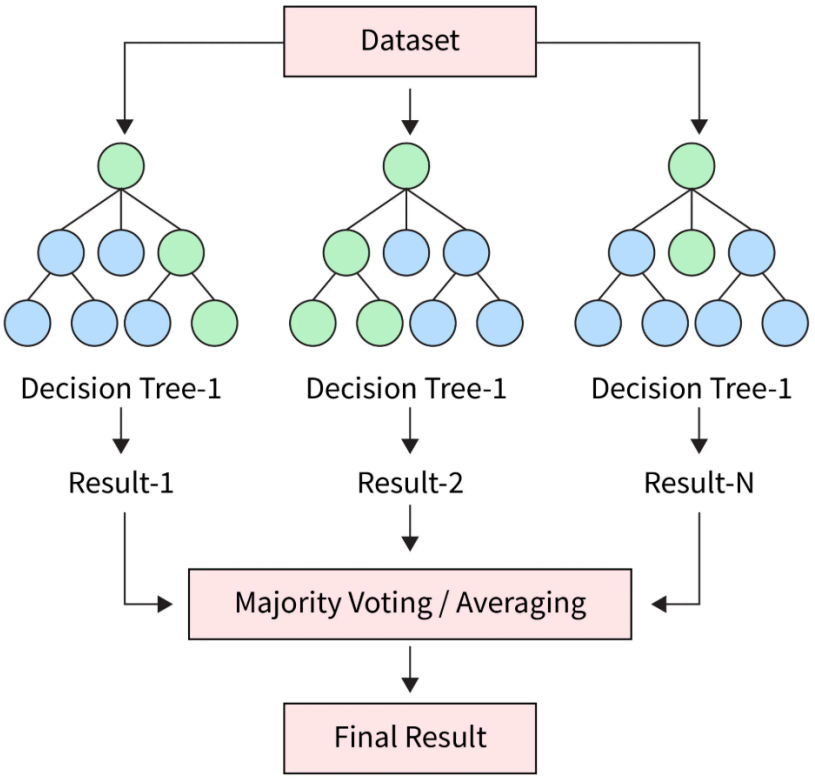
A Random Forest can be thought of as a team of simple rule-based submodels/mini-models that work together. Each mini-model learns a set of if-this then-that rules from the training data (for example, for cat and dog dataset if certain pixels are brighter and others are darker, the image is likely that of a 'cat'). The important bit is, every mini-model learns slightly different rules because it learns a slightly different mix of images and pays attention to different parts of each image. When you show the forest a new image, each mini-model makes a guess, and the forest picks the most common guess, like a simple vote.

Why this helps:

- A single rule-based model can be fragile, it might latch onto quirks of the training set.
- A team of slightly different models tends to be more reliable. Their mistakes often cancel out, so the majority vote is usually better than any one model on its own.

Why are we using it for our experiments:

- Because it's a solid non-deep-learning baseline: easy to train and often surprisingly decent.



A few knobs you'll see in the code (in simple terms):

- `n_estimators`: how many mini-models are in the team (more = steadier results but slower).
- `max_depth`: how long/complex each mini-model's rule chain can be (shorter can reduce overfitting).
- `random_state`: a seed so results are repeatable.
- `n_jobs=-1`: use all CPU cores to speed things up.

Note: Training on all 50,000 images can take a while on some systems. If needed, you can train on a smaller sample to speed things up.

Training Steps:

1. Initialize the model: We will use RandomForestClassifier from sklearn.ensemble. We can start with, say, 100 trees (`n_estimators=100`) and otherwise default parameters.
2. Train (fit) the model on the training data (`X_train_flat`, `y_train_flat`).
3. Compute the accuracy on the test set.

```
from sklearn.ensemble import RandomForestClassifier
  from sklearn.metrics import accuracy_score, confusion_matrix

  # Initialize Random Forest
  rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)

  # Train the model (this might take a few minutes)
  rf_clf.fit(X_train_flat, y_train_flat)
```

```
    # Make predictions on the test set
    y_pred_rf = rf_clf.predict(X_test_flat)

    # Calculate accuracy
    acc_rf = accuracy_score(y_test_flat, y_pred_rf)
    print(f"Random Forest test accuracy: {acc_rf:.4f}")
```

After running the above code, you'll get the accuracy of the Random Forest on the test data (for example, it might be in the range of 0.40-0.50 or so, i.e., 40-50% accuracy, since this is a tough task for a non-deep model). Next, let's compute the confusion matrix to see class-by-class performance.:

```
cm_rf = confusion_matrix(y_test_flat, y_pred_rf)
  print("Confusion matrix:\n", cm_rf)
```

The **confusion matrix** `cm_rf` is a 10$\times$ 10 matrix where the element at row i and column j indicates how many examples of class i were predicted by the model as class j. Ideally, most counts would be on the diagonal (correct predictions).

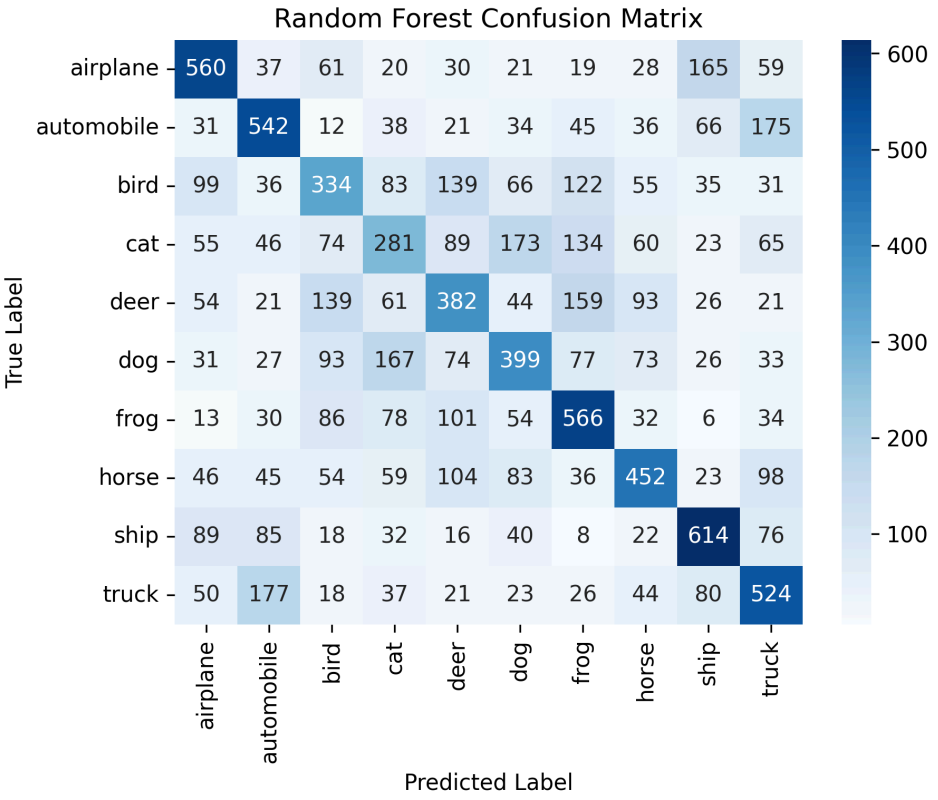
This raw matrix can be hard to read. It helps to visualize it with labels:

```
import seaborn as sns

  plt.figure(figsize=(6,5))
  sns.heatmap(cm_rf, annot=True, fmt="d", cmap="Blues",
              xticklabels=class_names, yticklabels=class_names)
  plt.title("Random Forest Confusion Matrix")
  plt.ylabel("True Label")
  plt.xlabel("Predicted Label")
  plt.show()
```

The heatmap shows which classes the Random Forest is good at (darker diagonal means many correct predictions) and where it confuses classes (off-diagonal cells). For instance, you might see confusion between similar-looking classes (e.g., cats vs. dogs). Overall, the Random Forest likely doesn't achieve very high accuracy on raw image pixels, but it gives us a baseline and some insight (notice which classes are hardest or easiest for it).

Question: Based on the confusion matrix, are there specific categories the Random Forest struggles with? Keep these observations in mind as we move to the neural network approach.


Random Forest Confusion Matrix
True Label
Predicted Label
airplane 560 37 61 20 30 21 19 28 165 59
automobile 31 542 12 38 21 34 45 36 66 175
bird 99 36 334 83 139 66 122 55 35 31
cat 55 46 74 281 89 173 134 60 23 65
deer 54 21 139 61 382 44 159 93 26 21
dog 31 27 93 167 74 399 77 73 26 33
frog 13 30 86 78 101 54 566 32 6 34
horse 46 45 54 59 104 83 36 452 23 98
ship 89 85 18 32 16 40 8 22 614 76
truck 50 177 18 37 21 23 26 44 80 524
airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck
600
500
400
300
200
100

Exercise 2: CIFAR-10 Classification with an Artificial Neural Network (ANN)

Now we will tackle the same task using a deep learning approach: a simple fully-connected neural network (ANN) built with Keras. This will involve constructing the network, training it on the images, and evaluating its performance. We expect the neural network to potentially perform better than the Random Forest on this image data, because it can learn more complex features. However, our ANN will be a fairly simple one (not a convolutional network), so we'll see how it fares.

Exercise 2a: Preparing Data for the Neural Network

We already loaded and flattened the CIFAR-10 images in Exercise 1 for the Random Forest. For the neural network, we'll keep using those flattened arrays but make two adjustments:

1. **Input shape**: Our network takes each image as a single vector of length 3072 (because $32 \times 32 \times 3 = 3072$). We will reuse the same flattened arrays you created earlier: `X_train_flat` and `X_test_flat`.
2. **Input scaling to [-1, 1]**: Neural networks often learn faster/better when the inputs are centered around zero. Since `X_train_flat` and `X_test_flat` are currently in the range [0, 1] (after dividing by 255 in Exercise 1), we will rescale them to [-1, 1] with a simple linear transform:

```
# Normalize the flattened images from [0, 1] to [-1, 1]
  X_train_flat = (X_train_flat - 0.5) * 2.0
  X_test_flat  = (X_test_flat  - 0.5) * 2.0
```

Exercise 2b: Building the Neural Network (Model Architecture)

We'll construct a feedforward neural network using Keras Sequential API. Sequential means our model will be a linear stack of layers. We need to decide on the architecture (how many layers and neurons). For this introductory exercise, we'll use a simple architecture:

- Input layer: 3072 input features (the flattened pixel values). We will specify this via the first layer's `input_shape`.
- Hidden Layer 1: Dense (fully-connected) layer with 512 neurons, using ReLU activation. This layer will take the 3072 inputs. We'll also apply a Dropout of 20% after this layer to help prevent overfitting (i.e., randomly drop 20% of the neurons' outputs during training).
- Hidden Layer 2: Dense layer with 256 neurons, ReLU activation.
- Hidden Layer 3: Dense layer with 128 neurons, ReLU activation.
- Output Layer: Dense layer with 10 neurons (one for each class), using softmax activation to output class probabilities.

This network has three hidden layers and and should be able to learn some intermediate representations. Now let's build this step by step:

```
from tensorflow.keras.models import Sequential
  from tensorflow.keras.layers import Dense, Dropout

  # 1. Initialize the Sequential model
  model = Sequential()

  # 2. Add the first hidden layer with 512 neurons and ReLU, and input
dimension of 3072
  model.add(Dense(units=512, activation='relu', input_shape=(3072,)))
  # Add dropout to reduce overfitting
  model.add(Dropout(0.3))

  # 3. Add second hidden layer with 256 neurons and ReLU
  model.add(Dense(units=256, activation='relu'))
  # Add dropout to reduce overfitting
  model.add(Dropout(0.3))

  # 4. Add third hidden layer with 128 neurons and ReLU
  model.add(Dense(units=128, activation='relu'))

  # 5. Add output layer with 10 neurons and softmax activation
  model.add(Dense(units=10, activation='softmax'))

  # Summarize the model architecture
  model.summary()
```

Hint: The model.summary() will print out the layers with their output shapes and number of parameters. Ensure that the final output shape is (None, 10), meaning for each input image we get 10 outputs (the probability for each class). The number of parameters might seem large, but this is expected for a network with these layer sizes (it will list weights and biases for each layer).

Exercise 2c: Compiling and Training the Neural Network

Before training, we need to compile the model with a loss function, optimizer, and metrics:

- For a multi-class classification (as in our case), we use `sparse_categorical_crossentropy` as the loss. This measures the difference between the predicted probabilities and the true distribution.
- We'll use Adam optimizer (a reasonable default for training neural nets).
- We'll track accuracy as our performance metric (percentage of images correctly classified).

Let's compile the model:

```
model.compile(optimizer='adam',
  loss='sparse_categorical_crossentropy',
```

```
    metrics=['accuracy'])
```

Now we are ready to train the model on the training data. We will train for a certain number of epochs (iterations over the entire training set). Keep an eye on the training and validation accuracy to see how the model is learning:

```
# Train the model for, say, 25 epochs with a validation split of 10%
  history = model.fit(X_train_flat, y_train,
                      epochs=25, batch_size=128,
                      validation_split=0.1, verbose=2)
```

Parameters:

- We use `validation_split=0.1` to set aside 10% of the training data for validation during training. Keras will report the validation loss and accuracy each epoch, which helps in monitoring overfitting.
- `batch_size=128` means the network updates its weights after every 128 images. (You can adjust the `batch_size` if needed.)
- `verbose=2` just makes the output less cluttered than the default.
- Training for 25 epochs might take a few minutes on CPU. If you have a GPU available, it will be faster. You can also start with fewer epochs (e.g., 10) to see results sooner, or more if you want to see if accuracy improves.

Watch the output each epoch. You'll see something like:

```
Epoch 1/25 - loss: ... - accuracy: ... - val_loss: ... - val_accuracy: ...
  Epoch 2/25 - ...
```

The training accuracy and loss should improve over epochs. The validation accuracy is important: if it stops improving or starts dropping while training accuracy keeps improving, the model might be overfitting.

Hint: If you notice overfitting (validation performance getting worse), you could introduce an early stopping mechanism. For example, using tensorflow.keras.callbacks.EarlyStopping to stop training when no improvement is seen for several epochs. This is optional but can save time. For now, our epoch count is modest, so it's okay to train fully.

Exercise 2d: Evaluating and Visualizing the ANN Performance

Now that we have trained our neural network, let's evaluate its performance on the test set (the 10,000 images that the model has not seen during training).

```
# Evaluate the model on the test set
  test_loss, test_accuracy = model.evaluate(X_test_flat, y_test, verbose=0)
```

```
print(f"Neural Network test accuracy: {test_accuracy:.4f}")
```

You should see the test accuracy printed. Compare this to the Random Forest accuracy from earlier. Typically, even this simple neural network might outperform the Random Forest on CIFAR-10, but since we did not use a convolutional neural network (which is more suitable for image tasks), the improvement might not be huge.

Let's gain more insights:

1. Plot the training history

We can visualize how the training and validation accuracy (or loss) evolved over epochs. This helps to diagnose underfitting or overfitting.

```
# Plot training & validation accuracy over epochs
  plt.figure(figsize=(6,4))
  plt.plot(history.history['accuracy'], label='Training Accuracy')
  plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
  plt.title('Training vs Validation Accuracy')
  plt.xlabel('Epoch')
  plt.ylabel('Accuracy')
  plt.legend()
  plt.show()
```

Similarly, you can plot the loss:

```
plt.figure(figsize=(6,4))
  plt.plot(history.history['loss'], label='Training Loss')
  plt.plot(history.history['val_loss'], label='Validation Loss')
  plt.title('Training vs Validation Loss')
  plt.xlabel('Epoch')
  plt.ylabel('Loss')
  plt.legend()
  plt.show()
```

Examine these plots. Ideally, training loss should decrease and validation loss should decrease too, but if validation loss starts rising while training loss keeps dropping, that's overfitting. If both training and validation loss plateau or validation is much lower than training, maybe the model could benefit from more capacity or training longer (underfitting).

2. Confusion Matrix for ANN

Just like we did for Random Forest, let's see where the ANN is getting things right or wrong.

```
# Predict classes for test set using the trained model
  y_pred_prob = model.predict(X_test_flat)             # probabilities for
```

```
each class
  y_pred_classes = np.argmax(y_pred_prob, axis=1)      # choose the class
with highest probability for each sample

  # Compute confusion matrix for ANN predictions
  cm_ann = confusion_matrix(y_test, y_pred_classes)

  plt.figure(figsize=(6,5))
  sns.heatmap(cm_ann, annot=True, fmt="d", cmap="Greens",
              xticklabels=class_names, yticklabels=class_names)
  plt.title("Neural Network Confusion Matrix")
  plt.ylabel("True Label")
  plt.xlabel("Predicted Label")
  plt.show()
```

Now compare this confusion matrix (for the ANN) to the one from the Random Forest. Are there improvements? You might observe that the neural network has higher numbers on the diagonal for some/many classes, indicating better accuracy. For example, it might be much better at distinguishing ships vs. airplanes, or cats vs. dogs, than the Random Forest was. Some classes might still be challenging (it's common for both models to confuse e.g. cats and dogs, since they can look similar at 32$\times$ 32 resolution).

3. [Optional] Sample Predictions

If you want, you can also pick a few test images and check what the ANN predicts vs the true label, just to see it in action.

```
# Pick a few random test images
  samples = [0, 1, 2, 3, 4]  # indices of test images to check (change as
desired)
  plt.figure(figsize=(8,2))
  for i, idx in enumerate(samples):
      plt.subplot(1, len(samples), i+1)
      plt.imshow(X_test[idx])  # show the image (remember X_test is still
32x32x3 in [0,1] range)
      true_label = class_names[int(y_test[idx])]
      pred_label = class_names[y_pred_classes[idx]]
      plt.title(f"T:{true_label}\nP:{pred_label}")
      plt.axis('off')
  plt.show()
```

This will display a few test images with "T:[true label]" and "P:[predicted label]". It's a nice sanity check to see what the model is getting right or wrong.

Discussion and Next Steps

We have trained two models on the CIFAR-10 image classification task:

- A Random Forest (traditional ML) and
- A simple 2-layer ANN.

Typically, the neural network should outperforms the Random Forest here, showing the power of even a basic deep learning model for image data. However, the performance might still not be extremely high because our network is simple. In practice, Convolutional Neural Networks (CNNs) are used for image classification as they are better at capturing spatial features.

[Optional] Tuning and Further Experiments

If you have more time or want to deepen your understanding, here are some optional exercises and ideas to try:

- Increase model complexity: Add more hidden layers or increase the number of neurons in each layer of the ANN. Does a deeper or larger network improve accuracy? (Be cautious of overfitting, more complex models may need more regularization or more training data.)
- Tune hyperparameters: Experiment with different hyperparameters for the ANN:
    - Training for more epochs (with an EarlyStopping callback to halt if no improvement).
    - Different batch sizes (e.g., 32 or 128) or learning rate (you can adjust Adam's learning rate).
    - Different activation functions or adding/removing dropout layers.

Try some of these and observe the effects. Always monitor training and validation performance to understand whether changes are helping or causing overfitting.

Conclusion

In this workshop, we saw how to approach an image classification problem using both classical machine learning and deep learning. This hands-on exercise has hopefully given you a clearer understanding of the workflow for building and evaluating models, as well as the considerations when dealing with image data.

Good job on getting through the workshop! Feel free to save this notebook for reference and continue experimenting with the models and techniques discussed.